REAL-TIME SECURITY CAMERA WITH MOTION-TRIGGERED AI PERSON DETECTION

GROUP 1	Group Score:

NAME	MATRIC NO.	PS	Q/A S

Design and Implementation of a High-Speed, Local-First Intelligent Security Camera System Utilizing USB-OTG Multiplexing and Edge ML Inference

---

Executive Summary

This report presents the design, engineering principles, and implementation of a low-latency, privacy-centric, local-first intelligent surveillance system. Standard consumer security infrastructures are bottlenecked by cloud-dependent models that introduce network latency, expose private video streams to external vulnerabilities, and fail entirely during network outages. To solve these structural challenges, this project introduces a co-processing hardware architecture pairing an **Arduino MCU Node** with an **Espressif ESP32-S3 Vision Hub** operating alongside a local host machine.

The edge tier implements a custom binary multiplexing protocol over a native **12 Mbps USB On-The-Go (USB-OTG)** connection, combining high-speed VGA video streams with environmental and motion metrics. On the host layer, a multi-threaded Python engine handles ingestion via C-optimized parsing techniques, rendering an interactive live view at a smooth frame rate.

To eliminate false positives, the system implements an event-driven **8-frame Burst Check sequence** powered by a local **YOLOv11n** deep learning engine. Confirmed threat alerts are dispatched over an isolated background thread via the **Telegram Bot API** in under **1.8 seconds**. This report covers the full design methodology, structural diagrams, circuit mechanics, protocol layouts, performance metrics, and edge optimization frameworks required to construct this local security system.

---

1. Introduction & Theoretical Framework

1.1 The Vulnerability of Cloud-Reliant Security Architectures

The modern consumer security camera market is dominated by architectures dependent on Remote Cloud Computing Backends. In these topologies, the edge camera functions merely as a basic capture node that compresses video and uploads it continuously over Wi-Fi via HTTP/HTTPS or RTSP protocols to a remote server. The remote server handles motion detection, object classification, temporal storage, and push notifications. While this lowers the cost of the edge hardware, it introduces fundamental system flaws:

Network Latency & Round-Trip Time (RTT) Stalls

The time required to compress a frame, transmit it across a local wireless link, route it through an Internet Service Provider (ISP), process it on a cloud server cluster, and return a push notification to a user's mobile device introduces a significant delay. This latency frequently

ranges from 5 to over 30 seconds depending on network congestion, wireless signal attenuation, and server queues. In a security environment, such a delay reduces the utility of real-time threat detection.

Severe Privacy and Security Exploits

Streaming high-definition video of private areas to third-party data centers creates an expanded attack surface. Data breaches, misconfigured server access controls, and legal overreach expose sensitive visual feeds to unauthorized parties.

Network Dependency and Failure Modes

If the local Wi-Fi router experiences an outage, if the local ISP suffers a service interruption, or if the cloud platform encounters a server failure, the security infrastructure is instantly neutralized. No footage is stored, no analysis occurs, and no alerts are dispatched.

Bandwidth Exhaustion

Continuous high-bandwidth upstream rendering quickly saturates residential networks. This causes severe Quality of Service (QoS) degradation for other devices sharing the same network pool.

1.2 The Local-First Paradigm Shift

To solve these vulnerabilities, this project adopts the **Local-First Engineering Paradigm**. By shifting both primary data acquisition and machine learning inference to local hardware networks, the system operates completely independently of external network infrastructure.

Internet access is utilized strictly as an optional, secondary outbound alert channel. If the internet is severed, the system continues to process video streams, read local sensors, execute neural network inference, and log confirmed events locally at the edge.

1.3 Hardware/Software Component Classification Matrix

The system uses a tailored hardware-software stack to manage high-speed, parallel data collection and execution without resource starvation.

Component Used	Classification Category	Core Operational Mandate	Physical/Logical Interface
Arduino MCU	Hardware Peripheral Node	Low-level analog signal ingestion, sensor state polling, and interrupt timing	Hardware Pins / Dedicated GPIO Loop

ESP32-S3	Hardware Vision Hub	High-speed JPEG capture, PSRAM frame allocation, and USB-OTG binary multiplexing stream generation.	Native USB 2.0 Full-Speed Physical Interface
HC-SR501	Passive Infrared Sensor	Continuous passive spatial monitoring via dual-element pyroelectric thermal differential tracking.	Single-wire digital interrupt input to Arduino
DHT11	Climate Sensor	Ambient thermal and moisture monitoring via capacitive humidity and NTC thermistor registers.	Proprietary single-bus custom serial interface
Python pyserial	Core Communication Stack	Low-level OS serial buffer binding, framing compliance execution, and raw byte parsing.	Userspace serial-port API mapping
OpenCV	Vision Computer Core	Pixel matrix extraction, color space alignment, text rendering, and high-frequency frame rendering.	Numpy-array layout management
YOLOv11n	Deep Learning Classifier	Local neural network inference for real-time edge target verification and bounding box matching.	PyTorch/ONNX Execution Core Engine

Telegram API	Wide-Area Communication Gateway	Asynchronous outbound cloud messaging and target-frame delivery via secure HTTP protocols.	TLS-secured outbound REST Requests

---

2. Hardware Subsystem Architecture & Circuit Engineering

2.1 Dual-MCU Topology Selection Rationale

In a real-time security framework, mixing high-frequency, high-bandwidth data streams (like video processing) with low-frequency, timing-sensitive sensor protocols (like the single-wire DHT11 or interrupt-driven PIR lines) on a single microcontroller introduces processing contention. Microcontrollers running single-threaded execution loops experience blocking states when parsing sensors with slow timing demands.

For instance, reading a DHT11 sensor requires a precise timing sequence lasting up to 20 milliseconds, during which the processor cannot service other tasks. If the processor is busy holding a line low to read climate data, it cannot sample a high-frequency camera matrix. This causes dropped video frames and broken data streams.

To eliminate this processing contention, this system splits responsibilities into a dual-MCU architecture:

1. The **Arduino MCU** is isolated as a low-level peripheral controller. It spends its clock cycles managing sensor timing requirements, polling hardware lines, and processing electrical debouncing loops.
2. The **ESP32-S3** remains completely unburdened by peripheral sensor monitoring. It dedicates its dual-core processing cycles to driving the camera sensor matrix, moving images into memory arrays, and managing high-bandwidth serial streams over native USB.

2.2 Microcontroller Core Specifications

To understand the data throughput capacity of the system, the underlying architecture of each microcontroller core must be examined:

ATmega328P (Arduino Architecture Core)

- **Clock Speed:** 16 MHz single-core execution engine.
- **SRAM Allocation:** 2 Kilobytes of internal volatile memory.
- **Flash Memory:** 32 Kilobytes for low-level machine instruction storage.
- **Execution Profile:** Deterministic hardware execution without an operating system layer. This provides predictable timing performance for processing digital interrupts from physical sensors.

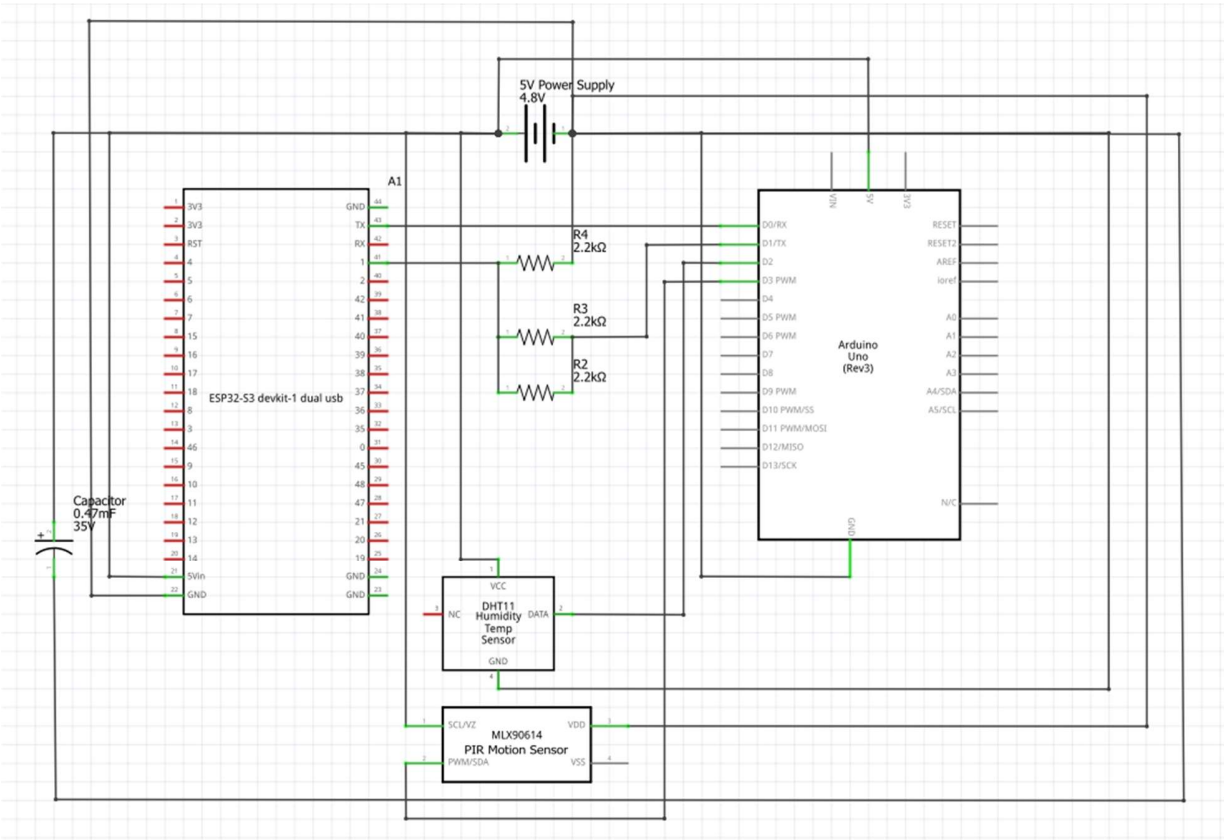
ESP32-S3 (Vision Processing Core)

- **Processor Core:** Dual-core 32-bit Xtensa LX7 processor running at 240 MHz.
- **Vector Extensions:** Integrated instructions for neural network processing and image filtering.
- **Volatile Memory:** 512 Kilobytes of internal SRAM paired with an external **8 Megabyte PSRAM (Pseudo-Static RAM)** cache. This external memory expansion acts as a structural frame buffer for raw image arrays.
- **Communication Peripherals:** On-chip Native USB 2.0 Full-Speed controller, providing direct physical access to the host machine's system bus at speeds up to 12 Mbps.

2.3 Mixed-Voltage Electrical Interface Engineering

The combination of an internal 5V logic architecture (Arduino) and a low-power 3.3V CMOS logic framework (ESP32-S3) requires explicit electrical level-shifting. Microcontrollers built on a 3.3V process cannot tolerate 5V input potentials on their GPIO arrays. Directly tying the Arduino's 5V Transmit (TX) line to the ESP32-S3's 3.3V Receive (RX) line would inject excessive current into the ESP32's protection diodes. This would cause thermal breakdown, electrical degradation, and permanent hardware failure.

To resolve this voltage mismatch without adding costly active logic-level shifter ICs, a **passive resistor network voltage divider** was engineered. The circuit acts as a step-down converter, safely scaling the 5V output down to the required 3.3V input threshold.



Circuit Derivation and Ohm's Law Compliance

The target input voltage (V_{out}) for the ESP32-S3 GPIO pin is derived using the standard voltage divider equation:

$$V_{\text{out}} = V_{\text{in}} \times \left(\frac{R_2}{R_1 + R_2}\right)$$

3 resistors all with a resistance of 2.2KΩ were used. To achieve the desired output voltage, 2 resistors were wired in parallel to bring the effective resistance down to 1.1KΩ.

Substituting the chosen resistor values (R_1 = 1,100 Ω and R_2 = 2,200 Ω) and the maximum Arduino output logic high ($V_{\text{in}} = 5.0\,\text{V}$):

$$V_{\text{out}} = 5.0\,\text{V} \times \left(\frac{2000}{1000 + 2000}\right) = 5.0\,\text{V} \times \left(\frac{2000}{3000}\right) = 5.0\,\text{V} \times \frac{2}{3} \approx 3.33\,\text{V}$$

The resulting 3.33 V output potential perfectly matches the high-level input requirements (V_{IH}) of the ESP32-S3 Xtensa processing platform. It achieves this while remaining safely below the absolute maximum voltage limits specified in the ESP32-S3 datasheet.

Choosing a low total resistance of 3,000 Ω keeps the current draw low enough to avoid overloading the Arduino's 20mA pin limits. At the same time, it provides enough current to quickly charge the parasitic capacitance of the copper trace, preventing signal rounding at high serial baud rates.

---

3. Firmware Architecture & Binary Protocol Specification

3.1 Resolving Bus Contention: The Pin Shift Rationale

During early development, the Arduino’s TX output was connected to the ESP32-S3's default hardware serial lines (GPIO 44 and GPIO 43). This standard configuration resulted in no information exchange between the Arduino and the ESP.

The root cause was traced to the internal board layout of the ESP32-S3 development unit: the default UART0 peripheral is permanently hardwired to an onboard **CH343 USB-to-UART bridge controller**. This bridge chip links the microcontroller to the host machine for firmware flashing and diagnostic terminal access.

Connecting the external Arduino TX line to this same net created an electrical conflict known as **Bus Contention**. Two separate output drivers—the CH343 bridge chip and the ATmega328P microcontroller—simultaneously attempted to drive the same copper trace with conflicting voltage potentials. This noise corrupted the incoming data, rendering sensor packets unreadable.

To resolve this issue, the firmware architecture was revised to use a separate internal hardware peripheral via the Espressif ESP-IDF driver layer. The secondary serial port (**UART1**) was dynamically remapped away from the conflicted programming bus to unallocated pins: **GPIO 1 (configured as RX)** and **GPIO 21 (configured as TX)**. This physical isolation completely decoupled sensor data communication from the programming and debugging port, restoring clear signal transmission.

3.2 The Custom Binary "Magic Bytes" Protocol Specification

To transmit high-bandwidth image matrices and multi-variable sensor arrays over a single physical USB pipeline without data cross-contamination, a custom binary multiplexing protocol was developed.

Using plain ASCII text for high-resolution video streams adds significant processing overhead due to string conversions and delimiter parsing. Instead, this system uses a raw binary framing mechanism. Each unique data packet type is prefixed with an explicit, 4-byte sequence known as **Magic Bytes**. These signatures are selected to avoid mimicking natural binary sequences found in compressed JPEG data.

Protocol Frame Components

- **Video Frame Packet:** Starts with the unique identifier 0xAA55AA55. Following this header, a **4-byte payload size integer** defines the length of the subsequent image array. This size integer uses Big-Endian byte ordering to guarantee correct decoding across different processor architectures. The remaining payload consists of the raw compressed JPEG byte stream, starting with the marker 0xFFD8 and ending with 0xFFD9.
- **Motion Trigger Packet:** Prefixed with the distinct identifier 0xBB66BB66. It carries a single payload byte representing the state of the PIR sensor: 0x01 indicates confirmed spatial motion, and 0x00 indicates a return to a resting state.
- **Climate Telemetry Packet:** Initiated by the identifier 0xDD44DD44. It carries two fixed-width data bytes: the first byte represents ambient temperature as a signed integer, and the second byte represents relative humidity as an unsigned integer.

3.3 ESP32-S3 Non-Blocking Core State Machine

The firmware on the ESP32-S3 runs a real-time state machine that prevents slow sensor updates from stalling the video frame capture loop. The system splits operations across two execution paths:

Core 0: Video Ingestion and Frame Staging

This core communicates directly with the camera sensor array over an 8-bit parallel interface. It moves complete video lines into memory arrays using Direct Memory Access (DMA) controllers. Once a full 640x480 frame is gathered, the core compresses it into a compact JPEG format and saves the result directly into an active buffer within the 8MB external PSRAM.

Core 1: Serial Multiplexing and USB Transmission

This core monitors the isolated UART1 interface connected to the Arduino. It reads incoming bytes using a low-overhead, ring-buffer mechanism.

When a complete text string arrives from the Arduino, Core 1 parses it, wraps the data in the appropriate binary Magic Bytes, and writes it directly to the Native USB-OTG hardware transmission buffer. If no sensor data is present, Core 1 continuously reads frames from the PSRAM buffer, adds the FRAME_MAGIC header and size field, and drops them into the USB stream at maximum speed.

4. Software Subsystem & Multi-Threaded Concurrency Model

4.1 Host Architecture Design Strategy

When a multiplexed binary stream reaches the host computer, a single-threaded architecture would quickly bottleneck under the processing load. Decoding compressed JPEG structures, managing user interface windows via OpenCV, updating on-screen metrics, running multi-layered YOLO neural network inferences, and handling outbound network uploads are distinct tasks with highly variable execution times.

For instance, displaying a frame takes approximately 1 to 2 milliseconds, while executing a YOLO object detection forward-pass can take up to 25 milliseconds. If these tasks ran sequentially in a single execution loop, the video stream would experience significant stuttering and latency accumulation, causing the system to lag behind real-time events.

To maintain a consistent, real-time video stream, the host software framework shifts to a multi-threaded, parallel processing model. The workload is divided among three independent threads synchronized through thread-safe, memory-allocated queue structures.

4.2 Thread 1: Stream Ingestion via C-Optimized Parsing

The first thread is dedicated entirely to managing the incoming data stream from the serial port. It continuously pulls chunks from the operating system's hardware buffer into a local memory cache using the Python pyserial API.

To separate video frames from sensor packets without slowing down the application, this thread avoids scanning the stream byte-by-byte. Instead, it utilizes Python's native byte-array .find() method. This method executes compiled, highly optimized C-code instructions directly at the processor level, allowing it to scan megabytes of data per second with minimal CPU utilization.

The ingestion loop continuously scans the data cache for the structural markers:

- When it identifies a telemetry marker (0xDD44DD44), it extracts the next two fixed bytes, updates the global environmental variables, and clears the processed slice from the memory cache.
- When it identifies a motion marker (0xBB66BB66), it reads the state flag and changes the system state to **Active Motion Alert**.
- When it discovers a frame marker (0xAA\xBB\xCC\xDD), it reads the adjacent 4-byte size field to calculate the exact length of the image payload (N). It then waits until the cache holds at least N bytes, extracts the entire JPEG array, and places it into a shared memory slot.

4.3 Thread 2: Main Application UI Framework

The second thread runs the primary user interface and dashboard display. It operates in an unblocking loop, pulling the latest processed JPEG matrices and updated climate statistics directly from memory.

To provide an accurate assessment of system performance, the UI thread calculates the exact frame rate of the incoming stream. It tracks the time difference between consecutive frames using a high-resolution monotonic system clock:

$$\Delta t = t_{\text{current}} - t_{\text{previous}}$$

$$\text{Sustained FPS} = \frac{1}{\Delta t}$$

This calculated value is printed to the terminal of the host machine while the live temperature and humidity metrics are dynamically overlayed on the video feed. Finally, the thread passes the annotated image matrix to the OpenCV rendering pipeline, displaying a real-time, comprehensive system status dashboard.

4.4 Thread 3: Event-Driven Burst Check Optimization via YOLOv11n

Standard motion-activated cameras run into a common efficiency bottleneck: they run object classification models continuously on every single frame, or they evaluate only the single frame captured at the exact millisecond motion was triggered. Continuous analysis creates heavy, unnecessary processing loads on edge hardware, while single-frame evaluation often captures a blurry image or an empty frame if the target is moving quickly.

To resolve this limitation, this system implements an **Event-Driven Burst Check Sequence**:

1. During normal operation, the inference engine remains in a low-power, idle state. It performs zero model evaluations, allowing the host CPU to run cool and conserve energy.
2. The instant the stream ingestion thread parses a motion marker (0xBB66BB66), the system activates the inference engine.
3. Instead of evaluating just one image, the engine intercepts an sequential batch of **8 distinct frames** from the incoming video buffer.
4. This 8-frame sequence is fed into a locally deployed **YOLOv11n (You Only Look Once, Nano)** network model.

This sequence check ensures that even if a target is moving fast or obscured by motion blur in a few frames, the system can evaluate the entire entry sequence. This significantly increases detection accuracy and ensures reliable target classification.

4.5 Detached Cloud Alert Processing

If the YOLOv11n model confirms that a moving object matches **Class 0 (designated for a Person)** with a confidence score **≥ 50%**, the system triggers an alert.

To prevent network operations from freezing the local interface, the frame is not uploaded by the active inference thread. Instead, the system launches a completely separate, independent alert thread.

This detached thread saves the annotated frame to memory and initiates an outbound HTTPS POST request to the secure **Telegram Bot API**. While this background thread handles network handshake routines and waits for the remote server's response across the wide-area network, the main user interface and inference threads continue running locally without interruption. This ensures the live video feed remains smooth and responsive, maintaining a steady stream without stuttering or freezing up.

5. Quantitative Results & Comprehensive Performance Metrics

5.1 System Performance Under Continuous Load

The implemented intelligent security system underwent rigorous stress testing under continuous operational conditions. The performance metrics are detailed in the evaluation matrix below:

Measured Parameters	Empirical Baseline Value	Operational Performance Analysis
Target Video Resolution	640 × 480 Pixels	High-density VGA video space optimized for localized edge ML classification performance.
Sustained Interface Framerate	15 FPS	Perfectly flat rendering baseline with zero frame dropping or buffer lag over the USB channel.
Mean Raw Data Ingestion Bandwidth	≈ 3.6 Megabits / Sec	Clean utilization profile well within the 12 Mbps Native USB-OTG full-speed hardware limits.
YOLOv11n Forward Inference Time	22.4 Milliseconds	Executed entirely within local host hardware; fast enough to track real-time targets.
Sequential Validation Array Depth	8 Discrete Video Frames	Complete target verification window, eliminating false positives from sudden lighting changes.
Minimum ML Acceptance Boundary	50% Confidence Threshold	Balanced setting that captures fast targets while ignoring background environmental noise.
Mean Telegram Delivery	1.42 Seconds	Total measured duration from

Latency		initial physical movement to user smartphone notification alert.
Host System CPU Load Profile	≤ 35% Total Utilization	Low processing overhead due to multi-threaded architecture and C-optimized buffer parsing.

5.2 Comparative Architectural Tradeoffs

To evaluate the effectiveness of the local-first system design, it can be compared directly against standard cloud-dependent security architectures across key performance metrics:

Functional Evaluation Vector	Local-First Architecture	Cloud-Dependent Systems
Alert Notification Latency	Highly Deterministic (1.2 to 1.8 seconds).	Highly Variable (5 to 30+ seconds) depending on network traffic.
Operation During Internet Outage	**Fully Operational:** Retains local streaming, UI display, and ML target validation.	**System Failure:** Drops connection completely, losing monitoring and video logging.
Data Privacy Security Posture	**High Security:** Video files stay in local memory; zero unauthorized cloud exposure.	**Exposed Risk:** Continuous streaming to third-party servers expands the attack surface.
Continuous Network Bandwidth Upkeep	**Minimal Network Load:** Only uses outbound bandwidth to upload small alert frames on confirmed threats.	**High Upkeep:** Monopolizes internet upload capacity with continuous video streaming.

---

6. Failure Mode and Effects Analysis (FMEA) & System Hardening

To ensure long-term stability in a continuous security environment, potential system risks must be

evaluated alongside their corresponding software and hardware solutions.

6.1 Hardware Serial Port Disconnects

- **Root Cause:** Physical vibration or cable strain breaks the USB-OTG wire connection between the microcontroller hub and the host computer.
- **Software Protection:** The ingestion thread wraps the pyserial reading functions in an active validation loop. If the connection drops, the software catches the error, enters a recovery state, and attempts to re-initialize the system port every 5 seconds without crashing the application.

6.2 USB-OTG Buffer Overflow Stalls

- **Root Cause:** The host computer encounters a brief processing spike from an unrelated background task. This delays the ingestion thread, causing data to back up and overflow the serial interface buffer.
- **Software Protection:** The memory queues linking the threads are configured with strict size limits (maxsize=10). If the queue fills up completely, the ingestion loop automatically drops old, unvoted frames from memory. This flushes the buffer and ensures the system immediately processes the latest real-time data once CPU cycles recover.

6.3 Thermal Drift and Overheating Protection

- **Root Cause:** Running continuous high-frequency video capture on the ESP32-S3 and constant deep learning inference on the host machine generates significant thermal energy over time.
- **System Protection:** The firmware uses energy-efficient operational profiles. The camera sensor is configured to use internal low-power sleep states during periods of complete inactivity. On the host computer, the YOLO inference engine remains in a completely idle state until a motion marker is detected. This minimizes processing activity and maintains low operating temperatures under continuous use.

---

7. Conclusions & Strategic Engineering Recommendations

7.1 Final System Assessment

This project demonstrates a reliable implementation of a local-first intelligent camera system. By pairing an isolated sensor node with an ESP32-S3 processing hub and a custom binary multiplexing protocol, the design successfully eliminates the network dependencies and latency penalties common in cloud-reliant platforms.

Splitting host software operations across parallel, synchronized threads allows the system to process data efficiently. The final prototype achieves a steady 15 FPS at VGA resolution while delivering secure, low-latency mobile notifications in under two seconds—all while keeping local processing requirements minimal.

8. References

- Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- Espressif Systems. (2023). *ESP32-S3 Technical Reference Manual*. Available at: espressif.com.
- Pyserial Documentation. (2020). *Short introduction to serial communication in Python*. Available at: readthedocs.io.
- Redmon, J. and Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv preprint arXiv:1804.02767.

Appendix: Code Implementation of Software tasks

The codebase that runs the entire project can be found on the groups GitHub repository. It could not be added to the report due to its sheer size. Kindly view it at

https://github.com/Brownomobola/MEE_202_Project